Fast exact overlap minimization for three-dimensional irregular strip packing of polyhedra

Ignacio Araya[a,*]

[a]*Escuela de Ingeniería Informática, Pontificia Universidad Católica de Valparaíso, Chile*

Abstract

We address the three-dimensional irregular strip packing problem: a set of polyhedra, possibly non-convex, must be placed by translation in a container with a fixed rectangular base so that the height is minimized. Pieces are represented exactly as unions of convex polytopes. Overlap between two convex parts is measured by the penetration depth of their relative position in the no-fit polytope (NFP), a quantity that, along any axis direction, is concave on an interval and zero outside it. Hence the position of a piece along an axis that minimizes its weighted overlap with all other pieces can be computed exactly by evaluating interval endpoints only. We embed this move in a strategic oscillation scheme with multiplicative guided local search weights and kicks. An ablation study shows that the weights and, above all, the speed of the separation are decisive. On the Merged instances of Egeblad et al. (2009), which use the same exact geometry, the method improves the best known heights by 4.5–9.6% within 120 seconds, and after 10 seconds its mean utilization exceeds that of the reference method after 600 seconds. It also improves the best known solutions of a classical convex instance and of the non-convex instances of Stoyan et al. (2004), one of which is solved to optimality, and on the small instances of Tollenaere et al. (2026) it matches 26 of the 27 proven optima and improves the best known solution of 32 of the 39 open instances. On the 36-piece instance of Liu et al. (2015) its mean height is 12% below the best voxel result. Finally, a linear-programming compaction over no-fit polytope facets with soft margins, followed by dyadic rounding, yields solutions whose feasibility is certified in exact rational arithmetic.

*Corresponding author

Email address: ignacio.araya@pucv.cl (Ignacio Araya)

Keywords: Cutting, Packing, 3D irregular packing, No-fit polytope, Guided local search, Linear programming

1. Introduction

Three-dimensional irregular packing arises in additive manufacturing (packing parts in the build volume of a printer), in the loading of non-rectangular goods, and in the design of compact assemblies. In its open-dimension variant, the *3D irregular strip packing problem*, a set of polyhedra must be placed without overlap inside a container with a fixed $W \times D$ base, and the height of the packing is to be minimized. We consider the translational version: pieces keep their orientation.

The literature on this problem is small compared with its two-dimensional counterpart, and it is split by the representation of the geometry. Exact approaches use the polyhedra directly: phi-functions and nonlinear programming (Stoyan et al., 2004, 2005; Romanova et al., 2018), the 3DNest local search of Egeblad et al. (2009), which minimizes the exact overlap volume along axis-parallel translations, constructive heuristics such as HAPE3D (Liu et al., 2015), and, recently, mixed-integer models based on no-fit polyhedra that solve small convex instances to optimality (Tollenaere et al., 2026). Approximate approaches discretize the pieces into voxels (Lamas-Fernandez et al., 2023); voxels make overlap tests trivial and scale well, but they overestimate the pieces and the quality of the solutions depends on the resolution.

In two dimensions, the most successful methods for irregular strip packing are overlap-minimization heuristics: the height (length) is decreased, the pieces are allowed to overlap, and a local search moves them until the overlap vanishes (Egeblad et al., 2007; Umetani et al., 2009; Imamichi et al., 2009; Elkeran, 2013; Gardeyn et al., 2025). The key ingredients are a cheap and informative overlap measure, moves that can be optimized exactly or quickly, and guided local search (GLS) weights to escape local minima. This paper brings these ingredients to three-dimensional polyhedra with an exact representation.

Contributions.

1. An exact and fast overlap-minimization move for polyhedra represented as unions of convex parts. Overlap is measured by the penetration depth in the NFP of each pair of convex parts; along an axis this

function is concave on an interval and zero outside it, so the optimal position of a piece along an axis is attained at an interval endpoint (Proposition 1). The move extends the directional moves of Umetani et al. (2009) to 3D and non-convex pieces and is much cheaper than the exact overlap volume of Egeblad et al. (2009).

2. A strategic oscillation with multiplicative GLS weights (Gardeyn et al., 2025) and kicks, and an ablation study showing that the weights and the speed of the separation are decisive, whereas a global LP compaction over NFP facets, in the spirit of Gomes and Oliveira (2006) in 2D, has no measurable effect on the quality.
3. New best known solutions on the Merged instances of Egeblad et al. (2009) and on classical instances of Stoyan et al. (2004, 2005), with a time-matched comparison against the exact method 3DNest, and a comparison with the optimal values and bounds of the mixed-integer models of Tollenaere et al. (2026).
4. Solutions with certified feasibility: a final LP compaction with soft margins and dyadic rounding produce layouts that are verified in exact rational arithmetic, without tolerances.
5. An exact convex decomposition of the non-convex (prismatic) pieces of Ikonen et al. (1997) used in the Merged instances, and the identification of inconsistencies in the public data.

Section 2 defines the problem and the geometric tools. Section 3 describes the algorithm, Section 4 the certification, and Section 5 the experiments.

2. Problem and geometry

2.1. Problem

We are given a container base $W \times D$ and n pieces $P_1, \dots, P_n$ (polyhedra, with multiplicities). Each piece is a union of convex polytopes, $P_i = \bigcup_{a \in \mathcal{A}_i} A_a$, expressed in a local frame where the minimum of its bounding box is the origin; w_i, d_i, h_i denote its bounding-box dimensions. A solution is a vector of translations $t_i = (x_i, y_i, z_i)$ such that

$$0 \le x_i \le W - w_i, \quad 0 \le y_i \le D - d_i, \quad z_i \ge 0, \qquad i = 1, \dots, n, \tag{1}$$

$$\operatorname{int}(P_i + t_i) \cap \operatorname{int}(P_j + t_j) = \emptyset, \qquad i < j, \tag{2}$$

and the objective is to minimize $H = \max_i (z_i + h_i)$. A trivial lower bound is $H_0 = \max\{\max_i h_i,\ \sum_i \operatorname{vol}(P_i)/(WD)\}$.

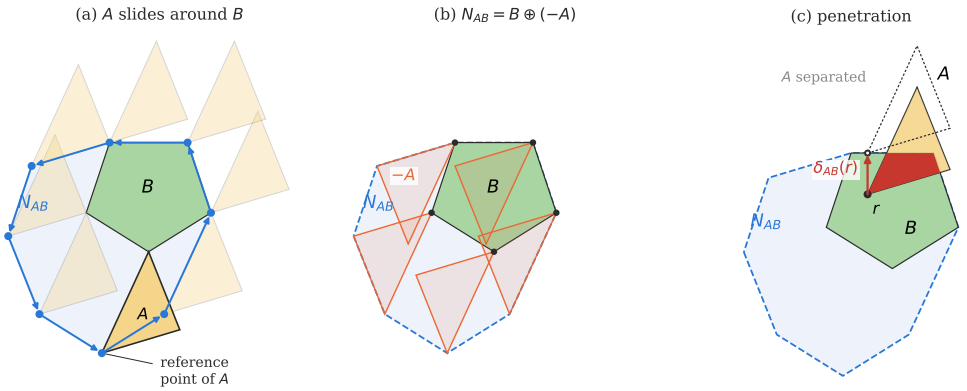


Figure 1: No-fit polytope and penetration in 2D. (a) When A slides around B keeping contact, its reference point (the minimum of its bounding box) traces the boundary of N_{AB}; the shaded copies of A touch B at the vertices of N_{AB}. (b) Equivalently, $N_{AB} = B \oplus (-A)$ is the convex hull of the copies of $-A$ placed at the vertices of B. (c) If r lies inside N_{AB} the parts overlap (red), and $\delta_{AB}(r)$ is the distance from r to the nearest facet: the shortest translation of A that separates the parts (dotted).

2.2. No-fit polytopes and penetration depth

For two convex parts A and B placed at t_A and t_B, their interiors intersect if and only if the relative position $r = t_A - t_B$ lies in the interior of the no-fit polytope

$$N_{AB} = B \oplus (-A) = \mathrm{conv}\{b - a : \ a \in V(A), \ b \in V(B)\}, \tag{3}$$

where $V(\cdot)$ is the vertex set (Bennell and Oliveira, 2008). We compute N_{AB} once per pair of parts as the convex hull of the vertex differences and store it in H-representation, $N_{AB} = \{r : \ n_f \cdot r \le d_f, \ f \in F_{AB}\}$, with unit normals $\|n_f\| = 1$.

Definition 1. *The* penetration *of A and B at relative position r is* $\delta_{AB}(r) = \max\{0, \ \min_{f \in F_{AB}}(d_f - n_f \cdot r)\}$.

For r inside N_{AB}, $\delta_{AB}(r)$ is the Euclidean distance from r to the boundary of N_{AB}, i.e., the length of the shortest translation that separates A and B (the translational penetration depth). It is zero if and only if the parts do not overlap. For two pieces we use $\delta_{ij}(t_i - t_j) = \sum_{a \in \mathcal{A}_i} \sum_{b \in \mathcal{A}_j} \delta_{ab}(t_i - t_j)$, where only pairs of parts whose bounding boxes intersect are evaluated. Figure 1 illustrates both definitions in two dimensions.

2.3. Exact minimization along an axis

Let all pieces but i be fixed, and let i move along axis e_k, $t_i(s) = t_i + (s - t_{ik})e_k$, with $s \in [0, U_k]$ (the container limits, using the current height H for the vertical axis). The weighted overlap of i is

$$\phi(s) = \sum_{j \neq i} w_{ij}\, \delta_{ij}\big(t_i(s) - t_j\big), \tag{4}$$

with GLS weights $w_{ij} \geq 1$.

Proposition 1. *Each term $\delta_{ab}(t_i(s) - t_j)$ is zero outside an interval (L_{ab}, U_{ab}) and concave inside it. Hence ϕ is concave on every interval between consecutive points of $\mathcal{B} = \{0, U_k\} \cup \{L_{ab}, U_{ab}\}$, and a minimizer of ϕ on $[0, U_k]$ is attained at a point of $\mathcal{B}$.*

Proof. Along the line, $d_f - n_f \cdot r(s) = c_f - n_{fk}s$ is affine. The set where all of them are positive is the open interval (L_{ab}, U_{ab}) with $L_{ab} = \max_{n_{fk}<0} c_f/n_{fk}$ and $U_{ab} = \min_{n_{fk}>0} c_f/n_{fk}$ (facets with $n_{fk} = 0$ only decide whether the interval is empty). Inside it, δ_{ab} is a minimum of affine functions, hence concave; outside it is zero. Between consecutive points of $\mathcal{B}$ every term is either concave or identically zero, so ϕ is concave there and attains its minimum at an endpoint. □

Figure 2 illustrates the proposition in two dimensions.

The move evaluates ϕ at the points of $\mathcal{B}$ only. With the interval of each term known, a term is evaluated only at the candidates inside it: sorting the candidates, each term contributes to a contiguous range found by binary search. Facets are stored as (c_f, n_{fk}) in contiguous arrays, so evaluating a term costs one fused multiply-add per facet. Ties are broken towards the smaller coordinate (gravity and walls).

3. Algorithm

Algorithm 1 summarizes the method. A greedy construction and an LP compaction give a first feasible solution S of height H. The search then tries to fit the pieces in a slightly lower container: the positions of S are *clamped* to a target height $h < H$: Clamp(S, h) moves every piece that exceeds h down until its top is at h, regardless of the other pieces. This is where overlaps appear, and MinimizeOverlap tries to remove them. If it succeeds, the

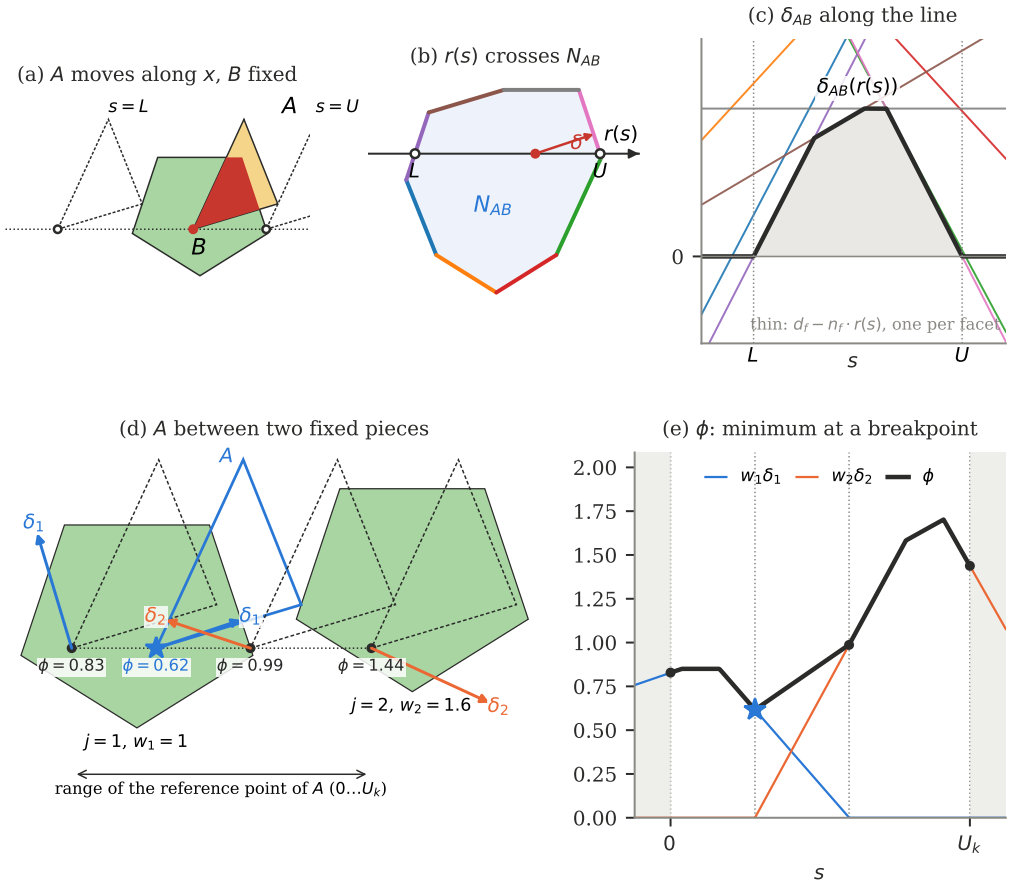


Figure 2: Proposition 1 in 2D, with the parts of Figure 1. (a) Part A moves along x with B fixed: it touches B at $s = L$ and $s = U$ (dashed) and overlaps it in between (red). (b) In the space of relative positions, $r(s)$ moves along a line that enters N_{AB} at L and leaves it at U. (c) For each facet (same colors as in (b)), $d_f - n_f \cdot r(s)$ is affine in s; the penetration is the lower envelope of these lines clipped at zero: zero outside (L, U) and concave inside. (d)–(e) A moves between two fixed pieces within the container range $[0, U_k]$. In (d), A is drawn at each breakpoint of (e) (0, U_k and the L, U of each term; dots: its reference point), with arrows for its penetration with each piece it overlaps (δ_1, δ_2) and the resulting $\phi = w_1\delta_1 + w_2\delta_2$ (weights $w_1 = 1$, $w_2 = 1.6$): the two central positions have similar penetrations but different ϕ. Between consecutive breakpoints the weighted overlap ϕ is a sum of concave functions, so its minimum (star, solid outline in (d)) is at a breakpoint, and only these points are evaluated.

Algorithm 1: Overview

```
S ← Greedy(); S ← Compact(S); H ← H(S);
h ← (1 − ρ)H; T ← Clamp(S, h); w ← 1;          // overlaps appear
while time remains do
    if MinimizeOverlap(T, h, w) then                // T is feasible
        T ← Compact(T);
        if H(T) < H then S ← T; H ← H(T);
        h ← (1 − ρ)H; T ← Clamp(S, h); w ← 1;
    else
        h ← h + ρH/4; T ← Clamp(T, h);        // keeps the overlaps
        if h ≥ H then
            h ← (1 − ρ)H; T ← Clamp(S, h); kick: move a random
              piece of T; w ← 1;
SeparateWithMargin(S); return S;
```

layout is feasible again: it is compacted, it becomes the new best solution, and the target is lowered; otherwise the target is raised slightly and, when it reaches H, a kick perturbs the best solution.

3.1. Greedy construction

Pieces are placed one at a time on top of the current load. For a piece, $g \times g$ starting points ($g = 4$) on the base are lifted to the current height and each descends by a sequence of small linear programs in the three translation variables: for every nearby placed part one NFP facet satisfied at the current point is selected (the one that allows the longest step in the descent direction), and $z + \sigma\,\mathrm{dist}_{\mathrm{wall}}(x, y)$ is minimized within a trust region; the procedure repeats until no improvement. Among all pieces and resulting positions, the greedy chooses the one maximizing $\alpha \log(V/V_{\max}) - \beta z/H_0 - \gamma \max(0, z + h - H_{\mathrm{cur}})/H_0$. For a given piece the criterion decreases with z, so the lowest of its positions is always preferred; the criterion thus decides which piece is placed, favoring large pieces placed low without increasing the height. Parameters ($\alpha = 1$, $\beta = 5.53$, $\gamma = 0.77$, $\sigma = 0.065$) were tuned with Optuna. Figure 3 illustrates the construction in two dimensions, and Algorithm 2 gives the pseudocode.

Algorithm 2: Greedy construction

function GREEDY*()*
  $\mathcal{L} \leftarrow \emptyset$ (placed pieces); $H_{\text{cur}} \leftarrow 0$;
  while *some piece is not placed* **do**
    foreach *type k with pieces left* **and** *each of $g \times g$ grid points (x, y) of the base* **do**
      $t \leftarrow \text{DESCEND}(k, (x, y, H_{\text{cur}}), \mathcal{L})$;
      $val \leftarrow \alpha \log(V_k / V_{\max}) - \beta z / H_0 - \gamma \max(0, z + h_k - H_{\text{cur}}) / H_0$;
    place the pair (k, t) of largest val in $\mathcal{L}$;
    $H_{\text{cur}} \leftarrow \max(H_{\text{cur}}, z + h_k)$;
  return $\mathcal{L}$;
function DESCEND*$(k, t, \mathcal{L})$*
  $c \leftarrow (\pm\sigma, \pm\sigma, 1)$; *// down and towards the nearest walls*
  repeat
    foreach *part a of k and placed part b of $\mathcal{L}$ that may meet a in the trust region* **do**
      select the facet f of N_{ab} satisfied at t that allows the longest step along $-c$;
    $t' \leftarrow \arg\min\{c \cdot u : \; n_f \cdot (u - t_b) \geq d_f \text{ (selected)}, \; \|u - t\|_\infty \leq \Delta, \; u \text{ in the container}, \; u_z \leq t_z\}$;
    if $c \cdot t' \geq c \cdot t$ **then break**;
    $t \leftarrow t'$;
  until *no improvement*;
  return t;

3.2. Global LP compaction

COMPACT is applied to feasible layouts: after the construction and after each successful separation. It solves a sequence of linear programs over all translations. Given the current layout, for every pair of parts that can touch within a trust region (each coordinate may move at most Δ, half of the smallest piece dimension) one NFP facet f satisfied by the current relative

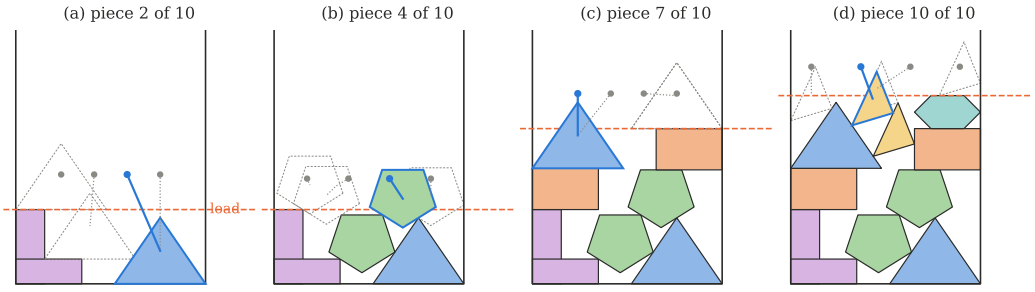


Figure 3: Greedy construction on a 2D example (10 pieces, one of them a union of two convex parts), computed with a 2D version of the method. In each step, a piece starts from $g = 4$ points on top of the current load (dots) and descends by a sequence of LPs over NFP facets, towards the floor and the nearest wall; dotted outlines: final positions of the other starts; blue: descent and position chosen by the greedy criterion.

position is selected and the constraint $n_f \cdot (t_i - t_j) \geq d_f$ is imposed. The LP

$$\begin{aligned} &\min\ H + \epsilon \sum_i z_i \\ &\text{s.t. } z_i + h_i \leq H, \quad \|t_i - t_i^0\|_\infty \leq \Delta, \quad i = 1, \ldots, n, \\ &\qquad \text{container bounds on } t_i, \qquad\qquad i = 1, \ldots, n, \\ &\qquad n_f \cdot (t_i - t_j) \geq d_f, \qquad\qquad \text{selected pairs of parts,} \end{aligned} \tag{5}$$

has a convex feasible region that contains the current layout. The facets are re-selected and the LP is solved again while H decreases. A solution is accepted only if it is feasible (the LP solution is checked for bounds and overlap). Compaction alternates with a local improvement in which every piece descends again with the others fixed and the pieces that define the height are tried at other positions (Algorithm 3).

3.3. Strategic oscillation

After a successful separation the target height is decreased by $\rho = 1\%$ of the best height and the best solution is clamped to it,

$$\textsc{Clamp}(T, h): \quad z_i \leftarrow \min\{z_i,\ h - h_i\} \text{ (and } x_i, y_i \text{ projected onto the container)}, \qquad i = 1, \ldots, n, \tag{6}$$

which creates overlaps among the pieces near the top. After a failure the target is increased by $\rho/4$ and the search continues from the overlapping

Algorithm 3: Compaction of a feasible layout

```
function Compact(T)
    T ← Improve(T);
    repeat
        T ← Improve(LPCompact(T));
    until H(T) does not decrease (at most 20 times);
    return T;
function LPCompact(T)
    repeat
        foreach pair of parts a ∈ A_i, b ∈ A_j that may meet in the
          trust region do
            select the facet f of N_ab satisfied by t_i − t_j that lets the
              higher piece descend the most;
        T′ ← solution of LP (5) with the selected facets;
        if T′ violates a bound or overlaps (LP tolerances) then
          return T;
        T ← T′;
    until H(T) does not decrease (at most 30 times);
    return T;
function Improve(T)
    foreach piece i in increasing order of z_i do
        t_i ← Descend(i, t_i, T \ {i});                    // others fixed
        if z_i + h_i = H(T) then
            move i to the lowest position generated as in Greedy, if
              it is lower;
    return T;
```

layout, with the GLS weights of the previous attempt. This scheme follows Egeblad et al. (2009) and Umetani et al. (2009).

3.4. Overlap minimization

MinimizeOverlap performs passes over the overlapping pieces in random order; for each piece the three axes are tried in random order with the exact move of Proposition 1. The pairwise penetrations are kept in a matrix and only the row of a moved piece is recomputed. When a pass

does not decrease (4), the GLS weights are updated multiplicatively as in Gardeyn et al. (2025): the weight of every overlapping pair is multiplied by $1.2 + 0.8\,\delta_{ij}/\max\delta$, and the other weights decay by a factor 0.95 (minimum 1). The procedure stops when no overlap remains (then verified with a separating-axis test) or after 500 passes. Weights persist along the oscillation and are reset after a successful separation or a kick (Algorithm 4).

Algorithm 4: Overlap minimization

function MinimizeOverlap(T, h, w)
  $P_{ij} \leftarrow \delta_{ij}(t_i - t_j)$ for all pairs; *// penetration matrix*
  for *pass* $= 1, \ldots, 500$ **do**
    $O \leftarrow \{(i,j) : P_{ij} > 0\}$;
    if $O = \emptyset$ **then return true** if the separating-axis test confirms it;
    gain $\leftarrow 0$;
    foreach *piece i of a pair of O, in random order* **do**
      foreach *axis $k \in \{x, y, z\}$ in random order* **do**
        $s^* \leftarrow \arg\min_{s \in \mathcal{B}} \phi(s)$ over the breakpoints $\mathcal{B}$ of Proposition 1, with the container $[0, W - w_i] \times [0, D - d_i] \times [0, h - h_i]$;
        gain $\leftarrow$ *gain* $+ \phi(t_{ik}) - \phi(s^*)$; $t_{ik} \leftarrow s^*$;
      recompute row i of P;
    if *gain* $= 0$ **then** *// local minimum: GLS*
      $w_{ij} \leftarrow \max(1,\, 0.95\, w_{ij})$ for $(i,j) \notin O$;
      $w_{ij} \leftarrow w_{ij}\,(1.2 + 0.8\, P_{ij}/\max_O P)$ for $(i,j) \in O$;
  return false;

Figure 4 shows an axis move and a complete separation, from a clamped layout, followed by the LP compaction on the 2D example.

3.5. Kicks

When the target reaches the best height, a kick restarts from the best solution clamped to $(1 - \rho)H$ and moves a random piece: with probability 1/2 it is reinserted at the lowest of the positions generated as in the greedy construction, below the target height (if there is none, it is swapped with a

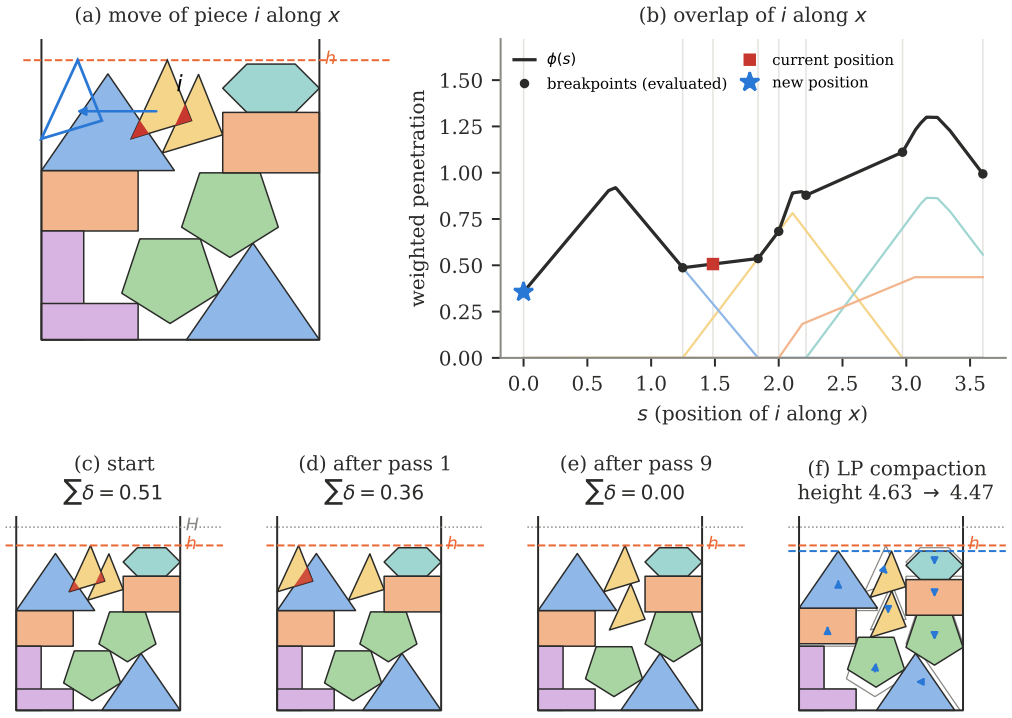


Figure 4: Overlap minimization on the 2D example of Figure 3. (a)–(b) Exact move of the most overlapped piece i along x: each term of ϕ (colored by piece) is zero outside an interval and concave inside it, so ϕ is evaluated only at the breakpoints and the best one is chosen (it reduces the overlap but does not remove it). (c)–(e) MinimizeOverlap on the greedy layout squeezed to the target height $h = 0.9H$ (overlaps in red): after 9 passes, with GLS weights, the layout is feasible. (f) The LP compaction that follows (gray: before) moves all pieces simultaneously and lowers the height below h.

piece of a different type), and otherwise it is moved to a random position in the container, leaving the overlap to the separation. The random moves are needed on small instances: with two pieces there is a single pair, so the GLS weights have no effect, and reinsertion alone keeps returning to the layout produced by the greedy order (Section 5.4).

4. Certified feasibility

LP solvers and floating-point NFPs leave overlaps of order 10^{-14} between touching pieces; tolerant verifiers accept them, but in exact arithmetic the pieces overlap. Note that safe bounds for linear programs (Neumaier and Shcherbina, 2004) certify the optimal value of the LP, not the feasibility of the point returned by the solver, so we certify the final layout instead.

SeparateWithMargin solves a last compaction LP in which each pair constraint becomes $n_f \cdot (t_i - t_j) \geq d_f + m_k$ and each container bound $x_i + u_i \leq W - w_i$ (and likewise in y), with slacks $m_k, u_i \in [0, \mu]$ ($\mu = 10^{-7}$, well above the LP tolerances) that are maximized while the height is kept. The slacks must be soft: rows of pieces that exactly fill the width of the container leave no room for a margin. In those rows the contact positions are typically sums of piece dimensions, which are representable; we then round every coordinate to the nearest multiple of 2^{-30}, an exact operation in binary floating point whose error is far smaller than μ (Algorithm 5).

Algorithm 5: Final separation with margins

function SeparateWithMargin*(S)*
  select one facet per pair of nearby parts as in LPCompact;
  solve $\min H - \frac{\epsilon'}{\mu}\big(\sum_k m_k + \sum_i (u_i + v_i)\big)$ s.t. the constraints of LP (5), with $n_f \cdot (t_i - t_j) \geq d_f + m_k$, $x_i + u_i \leq W - w_i$, $y_i + v_i \leq D - d_i$, $m_k, u_i, v_i \in [0, \mu]$ and a small $\epsilon' > 0$;
  if *the solution is feasible and H increases by at most $10^{-4}H$* **then** $S \leftarrow$ solution;
  $S' \leftarrow S$ with every coordinate rounded to the nearest multiple of 2^{-30};
  if *S' is feasible and $H(S') \leq (1 + 10^{-6})H(S)$* **then** $S \leftarrow S'$;
  return S; *// then certified in rational arithmetic*

The certificate is computed with rational arithmetic: vertices and translations are binary floating-point numbers, hence exact rationals. For containment all vertices are checked against the container; for every pair of convex parts with intersecting bounding boxes a separating axis is searched among the face normals of both parts and the cross products of their edges (computed exactly), accepting it if $\max_A n \cdot a \leq \min_B n \cdot b$. Of the 140 solutions of Table 2 (14 instances, 10 seeds), 139 were certified; in the remaining one (Stoyan 2005 Example 3) one pair of parts in contact overlaps by 1.2×10^{-10}. The height increases by 10^{-7}–10^{-6}.

5. Computational experiments

5.1. Setting

The method is implemented in C++ on top of the Metasolver framework; LPs are solved with SoPlex 8.1 (Wunderling, 1996). Experiments ran on two Intel Xeon E5-2620 v4 processors (2.1 GHz, 3.0 GHz turbo, 16 cores, 64 GB, Ubuntu 24.04, g++ 13.3), one thread per run, 14 runs in parallel. Each configuration was run with 10 seeds and a time limit of 120 s per instance.

Instances. We use the 3D instances of the ESICUP repository (EURO Special Interest Group on Cutting and Packing, 2025):

- *Stoyan 2005*, Examples 1–3 (7, 12 and 25 convex pieces) (Stoyan et al., 2005). In the public data, one piece (polytope7) has faces that use only 9 of its 10 vertices; we use its convex hull (volume 192.5), which matches the volume reported by Egeblad et al. (2009).

- *Stoyan 2004*, Examples 1–5 (20–50 non-convex pieces) (Stoyan et al., 2004), each defined as a union of convex polytopes, which we use directly as the decomposition.

- *Merged1–5* (Egeblad et al., 2009): i copies of 15 pieces, the 7 convex pieces of Stoyan 2005 and the 8 pieces of Ikonen et al. (1997) scaled by 4, in a container whose base is such that a layout with 50% utilization is a cube. Six Ikonen pieces are non-convex prisms; we decompose them exactly by triangulating the base polygon, merging triangles into convex polygons (Hertel and Mehlhorn, 1983), and extruding (a frame is split into four boxes; two pieces have polygons touching at a vertex). The volumes of the parts match those of the meshes.

- *Liu Experiment 1* (Liu et al., 2015): 36 pieces (8 tetrahedra, 8 cubes, 8 stars, 8 arrows and 4 rings) in a 20×20 container; the stars, arrows and rings are prisms and are decomposed exactly in the same way.

References. 3DNest (Egeblad et al., 2009) uses the same exact geometry; its average and maximum utilization over 10 runs of 600 s (one core of a 2.16 GHz Core Duo) are converted to heights. Lamas-Fernandez et al. (2023) report best and average heights of a voxel VNS; their voxelization overestimates the pieces (by 15% in volume for Merged1, as they note), so their heights are not directly comparable. For Stoyan 2004 the only exact reference is Stoyan et al. (2004).

5.2. Component analysis

Table 1 evaluates the components on the seven hardest instances (Stoyan 2004 Examples 1–5 and Stoyan 2005 Examples 2–3). Each row changes one element of the method. The effect of the speed of the separation is estimated by stopping the same runs at 24 s, which corresponds to moves five times slower.

The multiplicative weights and the speed of the separation are the decisive elements: with additive weights the mean heights increase on Stoyan 2004 Examples 3–5 and Stoyan 2005 Example 3 (e.g., from 53.60 to 55.27 on Stoyan 2004 Example 4), and five times slower moves lose most of the advantage. On these instances, kicks that only reinsert pieces are as good as the mixed kicks or slightly better; the random moves are needed on small instances (Section 5.4). The LP compaction has no significant effect: the differences are of about one standard error, slightly in favour of the variant without LP on Stoyan 2004. It is therefore not needed for the quality of the solutions, and the method can run without an LP solver; we keep a single LP at the end to obtain certifiable solutions (Section 4).

5.3. Comparison with the literature

Table 2 compares the method with the literature. On the instances that share the exact geometry of 3DNest (Stoyan 2005 Examples 2–3 and Merged1–5), the method obtains lower heights in six of seven cases; the exception is Stoyan 2005 Example 2 (19.13 versus 19.01). The improvement grows with the size of the instance: on Merged3–5 our best heights are 8–10% below the best known values, obtained by the voxel VNS with the finest resolution and four hours, and our mean heights after 120 s are lower than

Table 1: Mean gap of the height with respect to the best published value (Stoyan 2004 Examples 2–5 and Stoyan 2005 Examples 2–3) and mean height on Stoyan 2004 Example 1 (no published reference); 120 s, mean of 10 seeds. Negative is better.

Configuration	Stoyan 2004	Stoyan 2005	S2004 Ex1
Proposed method	−2.8%	−2.0%	53.41
Without LP compaction	−3.3%	−1.9%	53.30
Additive GLS weights ($w_{ij} \leftarrow w_{ij}+1$)	−1.3%	−1.5%	53.15
Kick: reinsertion only (no random moves)	−3.5%	−1.9%	53.50
Separation five times slower (runs stopped at 24 s)	−0.1%	−0.9%	54.82

those best values. On Stoyan 2004, the method improves the exact reference by a wide margin, and on Example 2 it reaches the optimum.

On Stoyan 2004 Example 2 our height, 29, equals the lower bound H_0 given by the tallest piece, so it is optimal for the public data. Stoyan 2004 Example 1 (80 pieces) has no published reference and is not included; the instance called Experiment 1 by Lamas-Fernandez et al. (2023) is the one of Liu et al. (2015), where our mean height is 12% below the best voxel height and also below that of HAPE3D, which allows rotations. For Stoyan 2005 Example 1, the utilization reported by Egeblad et al. (2009) converts to a height below the proven optimum with our data, so the instance seems to differ and we only compare with the optimum. Figure 5 shows two of the best solutions.

5.4. Comparison with exact models

Tollenaere et al. (2026) solve four series of small instances (2–20 items of the convex pieces of Stoyan 2005 and Stoyan 2004, of diamonds and of tetrahedra) with mixed-integer models based on no-fit polyhedra, with a time limit of one hour per model; they prove optimality for 27 of the 66 instances and report upper and lower bounds for the others. Table 3 summarizes our results (10 seeds, 30 s per instance). The method finds 26 of the 27 proven optima in its best run, 20 of them in every run; the exception is the 10-item instance of the Stoyan 2005 series (26.20 versus 26.05). On the 39 open instances it improves the best upper bound in 32 cases, by 5% on average, and matches it in most of the others. Before adding random moves to the kicks, the method missed the optimum of two 2-item instances (by 17% and

Table 2: Heights (lower is better). Exact: 3DNest (Egeblad et al., 2009) (best and mean of 10 runs of 600 s), optimum of Tollenaere et al. (2026) for Stoyan 2005 Example 1, and Stoyan et al. (2004) (St04). Voxels: VNS of Lamas-Fernandez et al. (2023) (best: standard resolution and 10 runs of 1 h for Stoyan, extra-fine resolution and 4 h for Merged; mean: standard resolution). Ours: mean (best) of 10 seeds, 120 s. *Optimal (equal to the lower bound H_0). †Value reported by Lamas-Fernandez et al. (2023); it is below the height of the tallest piece (29) in the public data. ‡HAPE3D (Liu et al., 2015), allowing 8 rotations. §Not exact: conservative convex decomposition (Section 5, Discussion); ours: 5 seeds of 20 min (15 min with a coarse decomposition and 5 min with a fine one); voxels: 10 runs of 1 h.

		Exact		Voxels		
Instance	n	best	mean	best	mean	Ours
Stoyan 2005 Ex1	7	17.46 (opt.)	–	–	–	17.47 (17.46)
Stoyan 2005 Ex2	12	**19.01**	19.83	21.23	21.70	19.13 (19.13)
Stoyan 2005 Ex3	25	28.83	29.82	29.76	31.53	**27.50 (27.22)**
Stoyan 2004 Ex2	20	32 (St04)	–	28.47†	29.40	**29.00 (29.00)***
Stoyan 2004 Ex3	30	49 (St04)	–	41.30	42.65	**39.90 (39.00)**
Stoyan 2004 Ex4	40	63.21 (St04)	–	54.83	59.52	**53.60 (52.35)**
Stoyan 2004 Ex5	50	78.7 (St04)	–	72.33	76.81	**66.81 (64.97)**
Merged1	15	17.00	17.45	17.13	18.67	**16.54 (16.04)**
Merged2	30	21.98	22.02	22.06	22.64	**21.04 (21.00)**
Merged3	45	24.04	24.97	24.32	26.26	**22.53 (22.02)**
Merged4	60	27.81	28.35	26.47	29.44	**25.31 (24.40)**
Merged5	75	30.13	31.31	29.24	31.12	**27.88 (26.44)**
Liu Exp. 1	36	31.2‡	–	34.27	35.37	**30.11 (29.52)**
Curved meshes (conservative decomposition)						
Chess§	32	–	–	**131**	**135.1**	145.0 (139.2)

28%): the greedy places the larger piece on the floor, the optimum needs the opposite arrangement, and neither the GLS weights (a single pair) nor reinsertion could escape.

5.5. Time-matched comparison

Hardware differs by about ten years. Figure 6 compares the mean utilization over time with that of 3DNest at 10, 60, 300 and 600 seconds. Our first solution is available after about 0.3 seconds (the greedy construction and compaction), and at 10 seconds our mean utilization already exceeds the mean utilization of 3DNest at 600 seconds on all five instances (44.3,

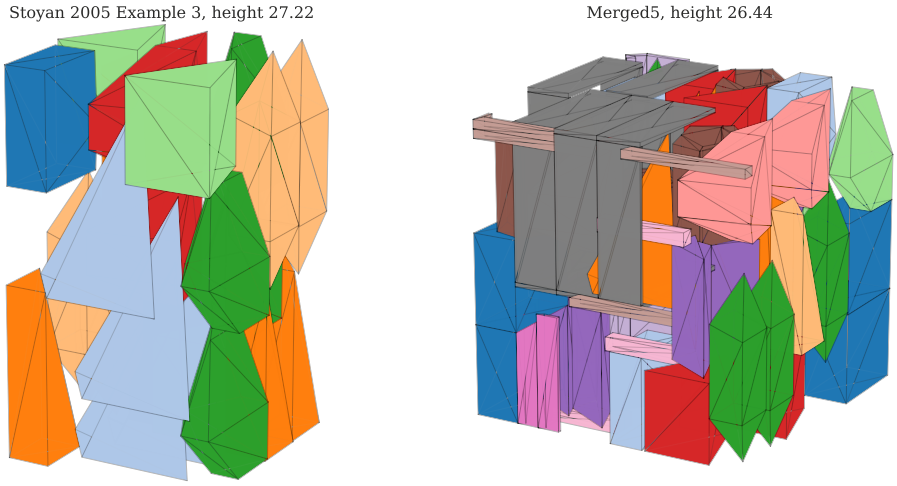


Figure 5: Best solutions of the final runs (certified) for Stoyan 2005 Example 3 and Merged5.

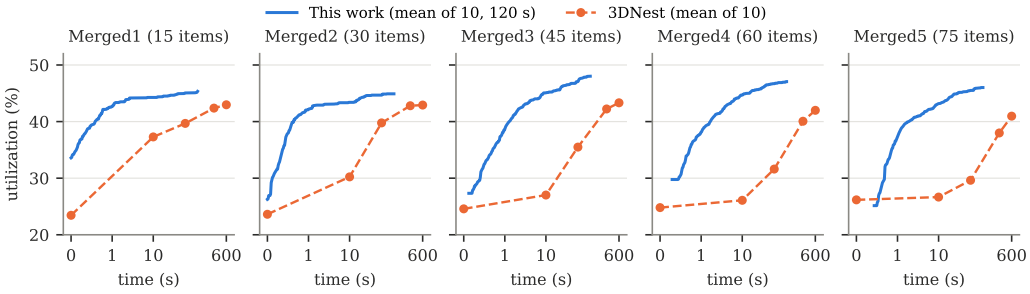


Figure 6: Mean utilization over time on the Merged instances (10 seeds), compared with 3DNest (Egeblad et al., 2009, Table 3) (10 runs).

43.4, 45.1, 44.7 and 43.2% versus 43.0, 42.9, 43.3, 42.0 and 41.0%). Even assuming that our machine is five times faster per core, the advantage holds at equal effective time.

5.6. Discussion

The results support a conclusion that agrees with recent 2D work (Gardeyn et al., 2025): the quality of an overlap-minimization heuristic is driven by how many separation attempts fit in the time budget, and therefore by the cost of evaluating and optimizing the overlap. The NFP-based penetration

Table 3: Instances of Tollenaere et al. (2026) (10 seeds, 30 s). Optima: instances with proven optimum, and number of them found in the best run and in all runs. Open: instances without proven optimum, number in which our best height is below the best upper bound of the models, and mean gap of our mean height with respect to that bound.

		Optima			Open	
Series	n	proven	best run	all runs	improved	gap
Diamonds	9	4	4	2	5 of 5	−4.5%
Stoyan 2004	19	9	9	6	9 of 10	−5.0%
Stoyan 2005	19	9	8	7	9 of 10	−4.7%
Tetrahedra	19	5	5	5	9 of 14	−1.7%
Total	66	27	26	20	32 of 39	

depth gives an exact, cheap and informative measure for unions of convex parts with few facets, which is the case of all the benchmark instances.

Realistic meshes. The cost of a move grows with the number of facets of the NFPs, so pieces with curved surfaces are the hard case. We tested the Chess instance of Lamas-Fernandez et al. (2023) (32 pieces, meshes of 700–2900 triangles). Since an exact convex decomposition of such meshes is impractical, we use a conservative one (the large convex hulls of the NFPs are computed with Qhull (Barber et al., 1996)): CoACD (Wei et al., 2022) cuts the mesh and takes the convex hull of each piece; the small gaps it leaves between parts are covered by scaling each part from its centroid, and containment is checked with interior sample points. With concavity threshold 0.05 (6–20 parts per piece) the union exceeds the volume of the piece by 4–12%, and with 0.2 (1–4 parts, reduced to at most 42 facet directions) by 38–78%. The solutions remain feasible for the real geometry. However, the NFPs between parts have hundreds of facets and a separation takes 20–35 s instead of a fraction of a second. In 20 minutes (5 seeds; Table 2) the best (mean) heights are 143.6 (150.5) with the fine decomposition, 151.2 (152.5) with the coarse one and 139.2 (145.0) with 15 minutes of coarse search followed by 5 minutes of refinement with the fine decomposition, against 131 (best of 10 runs of 1 h) and 135.1 (mean) for the voxel VNS of Lamas-Fernandez et al. (2023), whose cost does not depend on the complexity of the surface. Pseudo-blocks of two pieces did not help. The other realistic instance of Lamas-Fernandez et al. (2023), Engine (97 pieces with meshes of up to 342,000 triangles), was

not tested, for the same reason. Voxels and the proposed method are thus complementary: the latter is faster and exact on polyhedral pieces with few facets, the former scales better to curved meshes.

Limitations. Rotations are not considered. The method requires a convex decomposition of each piece; for the benchmark instances it is exact and small, while for curved meshes a conservative decomposition overestimates the pieces and, above all, makes the moves expensive. Reducing the cost of the NFPs of curved parts (e.g., hierarchical filtering with the hull of each piece, or conservative simplification of the NFPs) is left for future work.

6. Conclusions

We presented an overlap-minimization heuristic for 3D irregular strip packing that works with the exact geometry of polyhedra given as unions of convex parts. Its key element is an exact axis move based on penetration depths in no-fit polytopes, combined with multiplicative guided local search, strategic oscillation and kicks; a final LP compaction with margins makes the solutions certifiable in exact arithmetic. This simple method outperforms the previous methods, with exact and voxel geometry, on the benchmarks of polyhedral pieces, and matches almost all the proven optima of the small instances. On curved meshes, which require conservative decompositions with many facets, it remains behind voxel-based search. Future work includes cheaper NFP computations for curved parts, rotations, larger instances from additive manufacturing, and certified lower bounds.

References

Barber, C.B., Dobkin, D.P., Huhdanpaa, H., 1996. The quickhull algorithm for convex hulls. ACM Transactions on Mathematical Software 22, 469–483.

Bennell, J.A., Oliveira, J.F., 2008. The geometry of nesting problems: A tutorial. European Journal of Operational Research 184, 397–415.

Egeblad, J., Nielsen, B.K., Brazil, M., 2009. Translational packing of arbitrary polytopes. Computational Geometry 42, 269–288.

Egeblad, J., Nielsen, B.K., Odgaard, A., 2007. Fast neighborhood search for two- and three-dimensional nesting problems. European Journal of Operational Research 183, 1249–1266.

Elkeran, A., 2013. A new approach for sheet nesting problem using guided cuckoo search and pairwise clustering. European Journal of Operational Research 231, 757–769.

EURO Special Interest Group on Cutting and Packing, 2025. ESICUP datasets. `https://github.com/ESICUP/datasets`.

Gardeyn, J., Vanden Berghe, G., Wauters, T., 2025. An open-source heuristic to reboot 2D nesting research. arXiv:2509.13329.

Gomes, A.M., Oliveira, J.F., 2006. Solving irregular strip packing problems by hybridising simulated annealing and linear programming. European Journal of Operational Research 171, 811–829.

Hertel, S., Mehlhorn, K., 1983. Fast triangulation of simple polygons, in: Foundations of Computation Theory, pp. 207–218.

Ikonen, I., Biles, W.E., Kumar, A., Wissel, J.C., Ragade, R.K., 1997. A genetic algorithm for packing three-dimensional non-convex objects having cavities and holes, in: Proceedings of the 7th International Conference on Genetic Algorithms, pp. 591–598.

Imamichi, T., Yagiura, M., Nagamochi, H., 2009. An iterated local search algorithm based on nonlinear programming for the irregular strip packing problem. Discrete Optimization 6, 345–361.

Lamas-Fernandez, C., Bennell, J.A., Martinez-Sykora, A., 2023. Voxel-based solution approaches to the three-dimensional irregular packing problem. Operations Research 71, 1298–1317.

Liu, X., Liu, J.m., Cao, A.x., Yao, Z.l., 2015. HAPE3D—a new constructive algorithm for the 3D irregular packing problem. Frontiers of Information Technology & Electronic Engineering 16, 380–390.

Neumaier, A., Shcherbina, O., 2004. Safe bounds in linear and mixed-integer linear programming. Mathematical Programming 99, 283–296.

Romanova, T., Bennell, J., Stoyan, Y., Pankratov, A., 2018. Packing of concave polyhedra with continuous rotations using nonlinear optimisation. European Journal of Operational Research 268, 37–53.

Stoyan, Y.G., Gil, N.I., Pankratov, A., Scheithauer, G., 2004. Packing non-convex polytopes into a parallelepiped. Preprint MATH-NM-06-2004. Technische Universität Dresden.

Stoyan, Y.G., Gil, N.I., Scheithauer, G., Pankratov, A., Magdalina, I., 2005. Packing of convex polytopes into a parallelepiped. Optimization 54, 215–235.

Tollenaere, J., Martinez-Sykora, A., Wauters, T., 2026. Mixed-integer linear programming models for 3D irregular strip packing problems. European Journal of Operational Research 331, 365–380.

Umetani, S., Yagiura, M., Imahori, S., Imamichi, T., Nonobe, K., Ibaraki, T., 2009. Solving the irregular strip packing problem via guided local search for overlap minimization. International Transactions in Operational Research 16, 661–683.

Wei, X., Liu, M., Ling, Z., Su, H., 2022. Approximate convex decomposition for 3D meshes with collision-aware concavity and tree search. ACM Transactions on Graphics 41, 42:1–42:18.

Wunderling, R., 1996. Paralleler und objektorientierter Simplex-Algorithmus. Ph.D. thesis. Technische Universität Berlin.